Milestone 2 — Multi-Object Detection with YOLOv8

Study Group 9

73 object classes (OBJ001–OBJ073) · grid composites 2×2 to 5×5 · YOLOv8s detector

1. Introduction and Objective

Milestone 1 identified a single object in a 224×224 photo. Milestone 2 moves from classification to **object detection**: given an image that contains several objects, the model must report the **Object ID and the bounding-box location of every object** in the image. We cover all 73 classes (OBJ001–OBJ073); the YOLO class indices 0–72 map to these IDs in order.

Our approach has three parts. (i) Each single-object photo is given one tight bounding box with an open-vocabulary detector, verified by a classifier and a self-supervised feature model. (ii) These labelled photos are concatenated side-by-side into multi-object **grid images** of five sizes (2×2 up to 5×5), producing a detection dataset with correct boxes. (iii) A **YOLOv8s** detector is fine-tuned on this dataset and evaluated on a held-out test split. Building the multi-object images from single-object photos lets us generate large amounts of correctly-labelled detection data without any manual box drawing on cluttered scenes.

This report documents the data, the labelling method and how it was checked, the grid construction, the model and training configuration, the evaluation methodology, and the results (mAP@0.5, precision, recall) with example detections, followed by discussion, limitations and reproducibility notes.

2. Source Data (from Milestone 1)

The detector reuses the cleaned single-object photographs from Milestone 1: **7,370** images at 224×224 RGB — **7,000 object photos** across the 73 classes plus **370 background-only** photos. Cleaning honours EXIF orientation, converts to RGB, pads to a square with white borders (preserving aspect ratio), and removes exact duplicates by hashing both the normalised pixels and the re-encoded JPEG. The classes are split into train/val/test **before** any compositing, and this split is carried through to the detection dataset so that no source photo can appear in more than one split.

Each class contributes roughly 95 usable object photos. Objects appear at a range of scales and in cluttered real backgrounds, which is kept on purpose as part of the required variety.



Figure 1. A single-object photo with its auto-generated tight box (green) and inferred label — the unit that is pasted into the grids.

3. Bounding-Box Labelling and Quality Checks

The shared dataset provides only class labels (the folder name), so we generated one bounding box per single-object photo automatically and then checked a sample. The pipeline is:

3.1 Box generation

A pretrained open-vocabulary detector, **YOLOE-26**, proposes up to 12 candidate boxes per photo (class-agnostic NMS, weak confidence threshold so nothing is missed). Each candidate crop is scored two ways: (a) an **EfficientNet-B0 classifier** (trained in Milestone 1) gives the probability of the photo's known class, and (b) **DINOv2** self-supervised features measure how strongly the crop recurs across other photos of the same class (co-localisation). A short iterative selection combines detector confidence, classifier probability, a box-area prior and the cross-photo consistency to choose the box most likely to be the labelled object; photos still flagged as uncertain get a second, text-prompted YOLOE pass using the inferred object name. Background-only photos get empty label files.

3.2 Checking the labels

Because auto-generated boxes are not always correct, we (a) compared the prompt-free and text-prompted detections to catch disagreements, and (b) built a review queue of the highest-risk photos plus a random spot-check sample, inspecting them manually and correcting or excluding as needed. Photos were left out only when the object could not be labelled reliably.

3.3 Known dataset issues (handled in code)

Exact duplicate photos are removed before splitting, so no copy can leak across splits. The one non-224 image in OBJ003 is resized during cleaning. **OBJ054_017 is excluded** (it is not a photo of the object). In total **251 of 7,000 object photos (3.6%)** were left out — well under the 20% limit — for the reasons below; all 370 background photos were kept.

Exclusion reason	Photos
High-risk box, not manually reviewed	249
No visible target in the photo	2
Total excluded (of 7,000 object photos)	251 (3.6%)

4. Grid (Multi-Object) Dataset Construction

Multi-object images are built by placing 224×224 single-object photos side by side into a fixed grid. Every cell holds one object; the object's tight box is translated into the composite by adding the cell offset and normalising by the grid's pixel size. We build 714 images per grid size:

Grid	Objects / image	Image size (px)	Images
2×2	4	448 × 448	714
3×3	9	672 × 672	714
4×4	16	896 × 896	714
5×4	20	1120 × 896	714
5×5	25	1120 × 1120	714
Total	—	—	3,570

The objects in each composite are distinct classes, drawn with shuffled decks so occurrences stay balanced across the 73 classes (719–728 each). Boxes are copied from the single-object labels, so the detection labels are exact. The dataset splits as follows:

Split	Images	Object boxes
Train	2,500	37,000
Validation	535	7,918
Test	535	7,918
Total	3,570	52,836

Augmentation. Each tile is horizontally mirrored with probability 0.5 (its box mirrored too), and ±6% brightness/contrast jitter is applied to the whole composite with probability 0.45. At training time YOLOv8 adds mosaic, HSV colour jitter, translation, scaling and horizontal flips online. Random seed 20261013.

Source separation across splits. Training composites draw tiles only from training photos, and validation/test composites only from their own pools. We verified on the full manifest that all 6,749 distinct source photos belong to exactly one split — zero sources cross train/val/test.



Figure 2. A generated 5×5 grid composite with its ground-truth boxes (red) and Object IDs. Each cell is one single-object photo placed side by side.

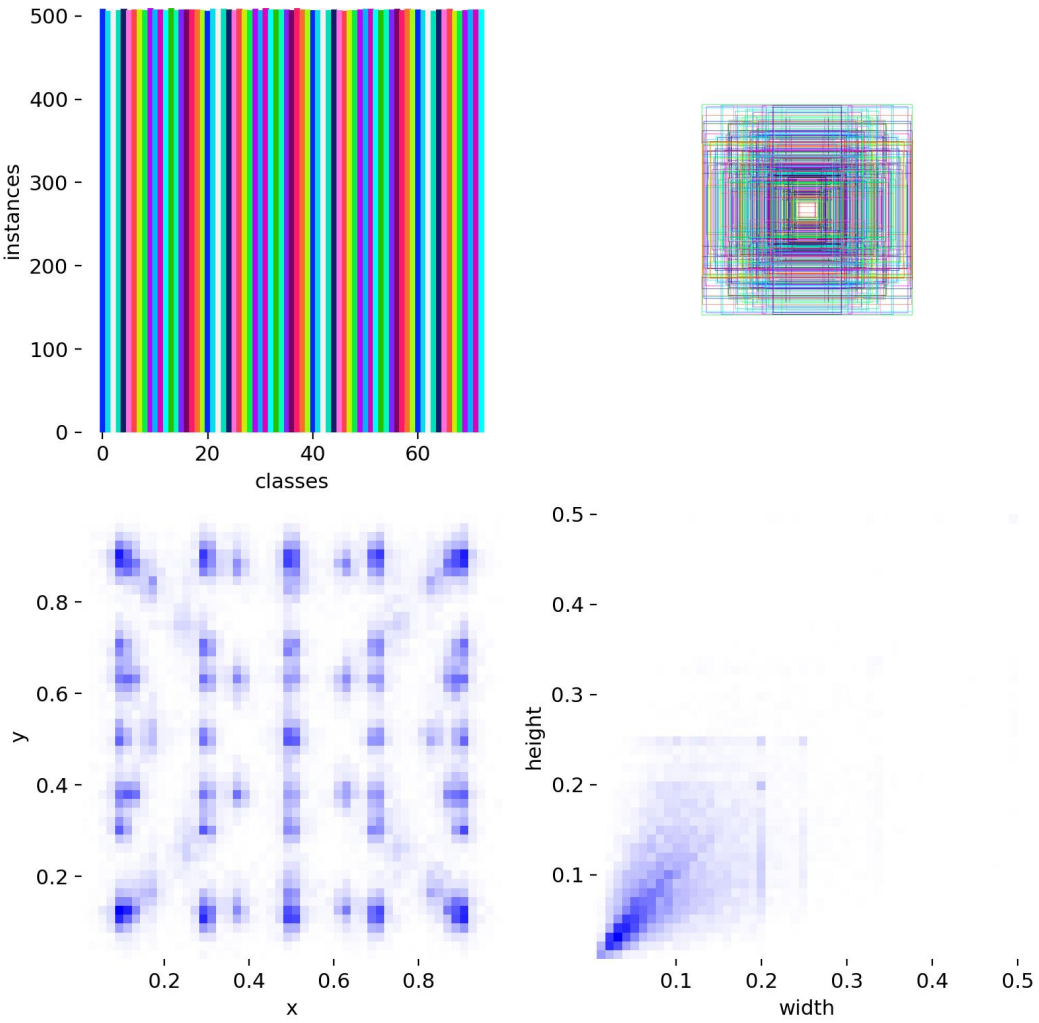


Figure 3. Label statistics of the training set: class balance, box centres, and box width/height distributions.

5. Model Architecture

We use **YOLOv8s** (small), a single-stage, anchor-free detector. A CSP-Darknet backbone extracts features at three scales; a PAN-FPN neck fuses them top-down and bottom-up; and a decoupled detection head predicts, at each scale, a class distribution (73 classes) and a distribution-focal-loss (DFL) box regression. Being anchor-free, it predicts box offsets directly rather than against preset anchors, which suits the wide range of object scales across the grids. The training objective sums a CIoU box loss, a binary cross-entropy classification loss, and the DFL term (weights 7.5 / 0.5 / 1.5).

Property	Value
Model	YOLOv8s (COCO-pretrained, fine-tuned)
Detection head classes	73 (OBJ001–OBJ073)
Layers (fused)	72
Parameters	11.2 M
Compute	28.6 GFLOPs @ 640 px

6. Training Setup

We fine-tune COCO-pretrained YOLOv8s with a fresh 73-class head. Local Apple-MPS training failed on a PyTorch backward-pass bug (an internal non-contiguous `view` during gradient computation), so training was run on a **Kaggle Tesla T4 GPU**. The input size is **896 px**, chosen close to the largest grids so that objects in 5×5 composites are not shrunk too far when the detector resizes the image. Early stopping (patience 10) ended training at epoch 34; the best checkpoint is epoch 24.

Hyperparameter	Value	Hyperparameter	Value
Base weights	yolov8s.pt	Epochs (patience)	50 (10)
Image size	896	Batch size	16
Optimizer	AdamW (auto)	Initial LR	1.3e-4
LR schedule	cosine	Final LR factor	0.01
Momentum	0.937	Weight decay	5e-4
Warm-up epochs	3	Box / cls / dfl	7.5 / 0.5 / 1.5
Mosaic	1.0	HSV (h,s,v)	0.015, 0.7, 0.4
Translate / scale	0.1 / 0.5	Flip LR	0.5
Seed	20261003	Mixed precision	on (GPU)



Figure 4. A training batch showing YOLOv8's online augmentation (mosaic, colour jitter, flips) applied to the grid composites.

7. Evaluation Methodology

A prediction is a true positive when it has the correct Object ID and its box overlaps a ground-truth box by at least the required Intersection-over-Union (IoU). **Precision** = TP / (all predictions) measures how many predictions are right; **recall** = TP / (all ground-truth objects) measures how many objects are found. **mAP@0.5** is the mean Average Precision across classes at an IoU threshold of 0.5, and **mAP@0.5:0.95** averages AP over IoU thresholds from 0.50 to 0.95 in steps of 0.05 (a stricter localisation measure). All figures below are on the 535 held-out **test** images (7,918 objects), which are never used for training or checkpoint selection.

8. Results

The detector reaches **mAP@0.5 = 0.867** on the test set, with precision 0.878 and recall 0.851. Validation at the selected checkpoint was comparable, indicating no over-fitting to the validation split.

Test metric	Value
mAP@0.5	0.867
Precision	0.878
Recall	0.851
mAP@0.5:0.95	0.770

Validation (best ep.)	Value
mAP@0.5	0.860
Precision	0.877
Recall	0.837
mAP@0.5:0.95	0.771

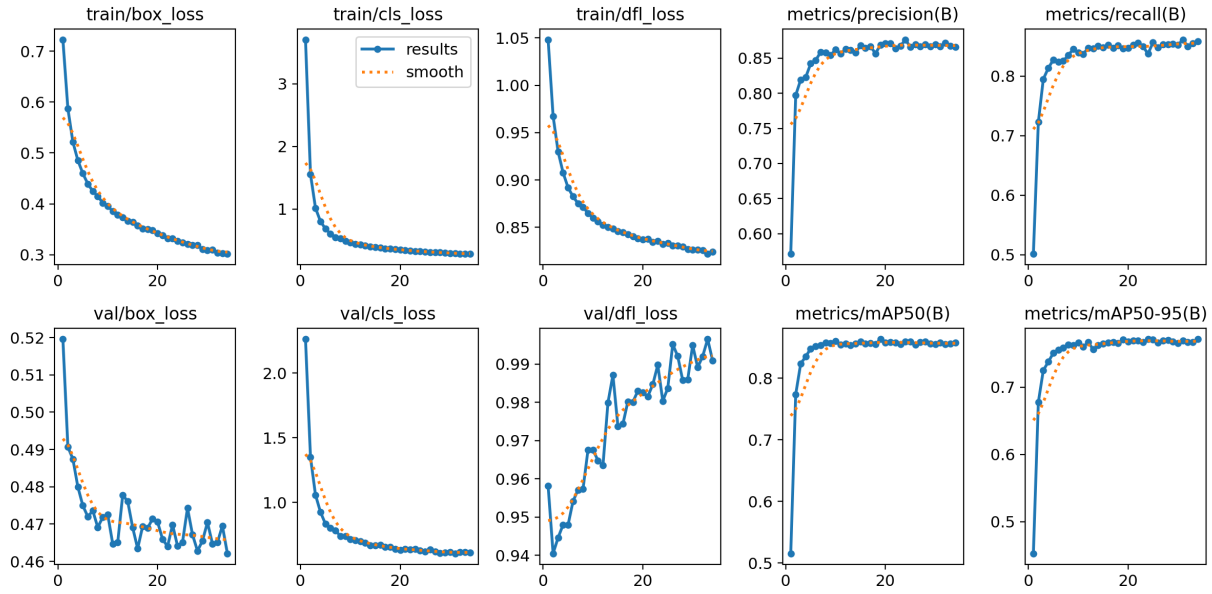


Figure 5. Training and validation curves over 34 epochs: box/cls/dfl losses and validation precision, recall, mAP@0.5 and mAP@0.5:0.95.

8.1 Example detections

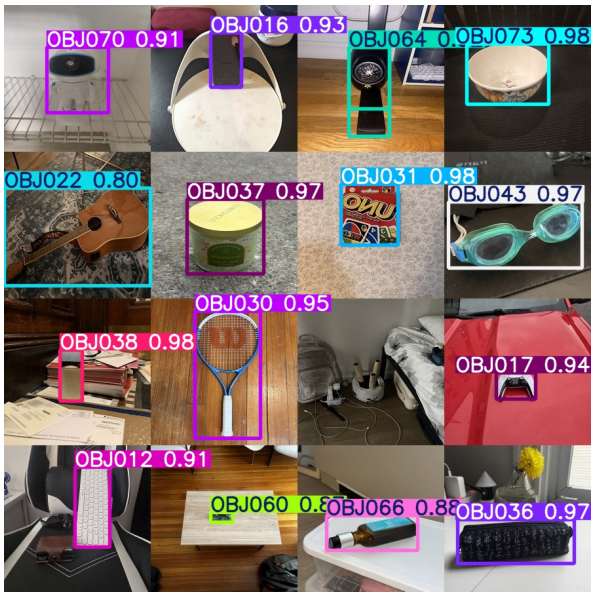


Figure 6. 4×4 grid — predicted Object IDs + boxes.



Figure 7. 5×5 grid (25 objects) — predicted Object IDs + boxes.

8.2 Precision–recall and prediction vs. ground truth

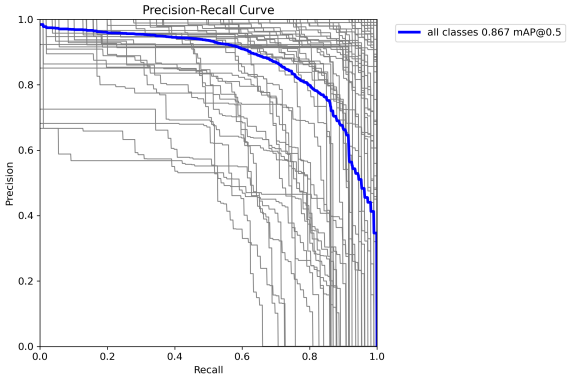


Figure 8. Test precision–recall curve (all classes).

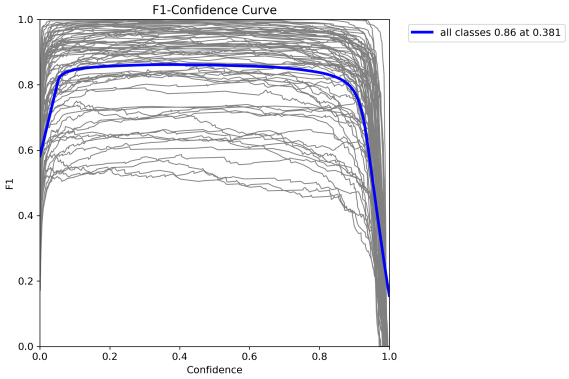


Figure 9. Test F1 vs. confidence (all classes).



Figure 10. Model predictions on a test batch.



Figure 11. Ground-truth labels for the same batch.

9. Discussion and Error Analysis

Qualitatively the detector is strong: in the example grids almost every object is found with the correct ID at high confidence (typically 0.90–0.98), with tight boxes. Because each grid cell is a clean, non-overlapping 224×224 crop, localisation is easy; the main difficulty is **classification among 73 visually similar everyday objects**, which is where the remaining errors concentrate.

Performance is slightly lower on the denser grids (4×4, 5×4, 5×5): with up to 25 objects per image each object occupies a smaller fraction of the resized input, so faint or heavily-cluttered tiles are occasionally missed (lowering recall) or confused with a similar class. Choosing an input size of 896 px rather than the default 640 px directly mitigates this by keeping the per-object resolution higher. Precision stays high throughout, meaning the boxes the model does emit are usually correct.

A second-order effect is that the model was fine-tuned from COCO, whose categories do not include these 73 objects; the gains over epochs (Figure 5) come almost entirely from learning the new classes, and the curves plateau once the classes are learned, which is why early stopping triggers at epoch 34.

10. Limitations and Future Work

Auto-generated labels. Boxes come from an automatic pipeline; although checked on a sample, a small fraction may be loose or wrong, which caps achievable mAP. **Grid realism.** Objects in a grid are placed in clean, non-overlapping cells, so the model is not tested on occlusion or objects at arbitrary positions and scales within a single scene. **Single configuration.** We trained one model size (YOLOv8s) at 896 px; training at 1120 px, or with a larger backbone, would likely raise mAP on the 5×5 grids at the cost of compute. Future work: add genuinely multi-object photographs, tighten labels with a second detector for agreement, and sweep input size and model scale.

11. Reproducibility

Code: **https://github.com/Sneha0504/ie7615-object-identification**. Datasets and weights are large and are not committed. The pipeline is: (1) `annotate_single_object.py` generates the single-object boxes; (2) `generate_grid_dataset.py` builds the grid detection dataset from them; (3) `kaggle_train_grids.py` trains YOLOv8s on a GPU; and (4) `run_live_demo.py` detects objects in any new image(s), printing each Object ID with its box and saving annotated outputs. Fixed seeds are used throughout.

12. Conclusion

By reusing single-object boxes and concatenating them into grid composites of five sizes (2×2 to 5×5), we produced a correctly-labelled multi-object detection dataset and trained a YOLOv8s detector that reports the ID and location of every object in an image. On a held-out test set it achieves mAP@0.5 = 0.867, precision 0.878 and recall 0.851, with reliable, high-confidence detections across all grid sizes — meeting the Milestone-2 goal of detecting multiple objects and returning their IDs and locations.

Appendix A. Full Training Configuration

Key Ultralytics settings for the final run (others left at library defaults):

Setting	Value
task / mode	detect / train
model	yolov8s.pt (COCO-pretrained)
data	grid dataset (73 classes)
epochs / patience	50 / 10 (stopped at 34)
imgsz	896
batch	16
device	Kaggle Tesla T4 GPU (CUDA)
optimizer	AdamW (auto)
lr0 / lrf	0.00013 / 0.01
momentum / weight_decay	0.937 / 0.0005
warmup_epochs	3.0
cos_lr	true
box / cls / dfl gains	7.5 / 0.5 / 1.5
hsv_h / hsv_s / hsv_v	0.015 / 0.7 / 0.4
translate / scale	0.1 / 0.5
fliplr / mosaic	0.5 / 1.0
amp / seed	true / 20261003

Appendix B. Additional Example Detections

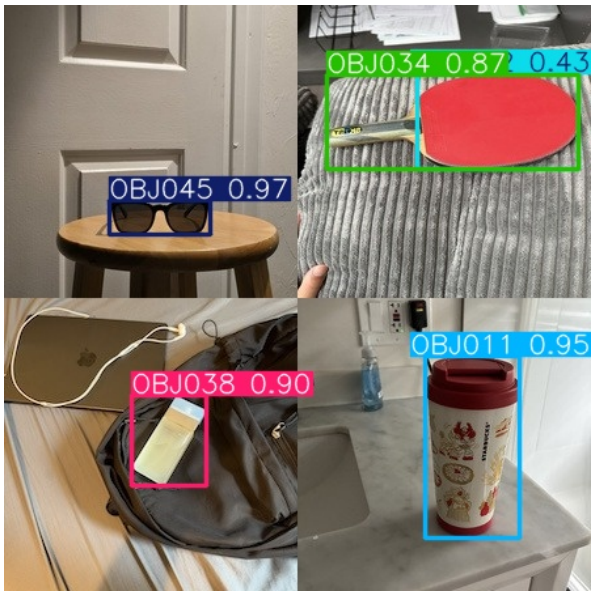


Figure B1. Test grid detections.



Figure B2. Test grid detections.



Figure B3. Test grid detections.

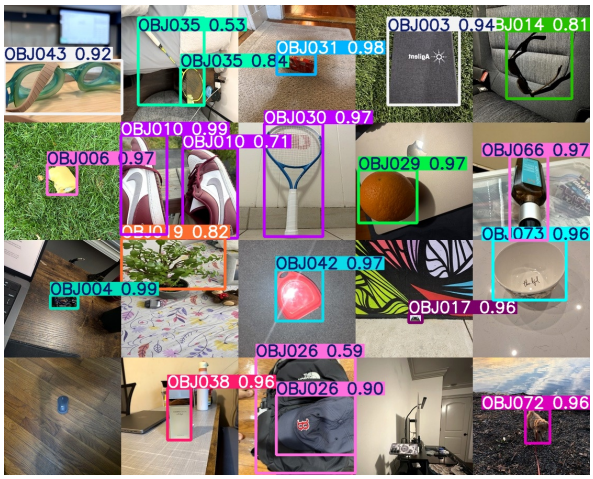


Figure B4. Test grid detections.

Appendix C. Object IDs (73 classes)

YOLO class indices 0–72 map to OBJ001–OBJ073 in order:

OBJ001	OBJ002	OBJ003	OBJ004	OBJ005	OBJ006	OBJ007	OBJ008
OBJ009	OBJ010	OBJ011	OBJ012	OBJ013	OBJ014	OBJ015	OBJ016
OBJ017	OBJ018	OBJ019	OBJ020	OBJ021	OBJ022	OBJ023	OBJ024
OBJ025	OBJ026	OBJ027	OBJ028	OBJ029	OBJ030	OBJ031	OBJ032
OBJ033	OBJ034	OBJ035	OBJ036	OBJ037	OBJ038	OBJ039	OBJ040
OBJ041	OBJ042	OBJ043	OBJ044	OBJ045	OBJ046	OBJ047	OBJ048
OBJ049	OBJ050	OBJ051	OBJ052	OBJ053	OBJ054	OBJ055	OBJ056
OBJ057	OBJ058	OBJ059	OBJ060	OBJ061	OBJ062	OBJ063	OBJ064
OBJ065	OBJ066	OBJ067	OBJ068	OBJ069	OBJ070	OBJ071	OBJ072
OBJ073							